QONTOS: Quantum Orchestrated Network for Transformative Optimization Systems

Technical Whitepaper v1.1

QONTOS Quantum Technologies
Zhyra Quantum Research Institute (ZQRI)
Advanced Quantum Systems Division
Abu Dhabi, UAE
research@zhyra.xyz — qontos@zhyra.xyz

***Abstract*—QONTOS is a distributed quantum computing orchestration platform designed to abstract the complexity of multi-backend quantum execution behind a single, unified API. The platform accepts quantum circuits in multiple formats (OpenQASM 2.0/3.0, Qiskit, PennyLane), partitions them across available quantum backends (IBM Quantum, Amazon Braket, local simulators), schedules execution for fidelity and cost, aggregates distributed results, and provides cryptographic execution proofs without requiring the end user to manage provider-specific APIs, circuit transformations, or result reconciliation.**

The QONTOS platform is built on a software-first, hardware-agnostic philosophy. The core orchestration pipeline—implemented as a 6-stage Celery task chain (ingest, partition, schedule, execute, aggregate, verify) backed by PostgreSQL and Redis—is implemented and benchmarked on Qiskit Aer simulators today. Around that platform, QONTOS maintains a broader modular quantum research program covering scaled architecture, qubit platform assumptions, error correction, photonic interconnects, cryogenic infrastructure, digital twins, and benchmark methodology.

This whitepaper documents both layers clearly: current implementation-grounded platform capability and the scenario-based research program that extends QONTOS toward modular hardware systems. Simulator-backed results are labeled as measured platform evidence. Larger hardware, logical-qubit, and million-qubit outcomes are treated as base, aggressive, or stretch program targets derived from literature, modeling, and internal architecture planning.

***Index Terms*—quantum computing, orchestration, modular architecture, fault-tolerant computing, error correction, multi-provider scheduling, distributed quantum execution**

I. Introduction

A. The Quantum Computing Challenge

Quantum computing promises exponential speedups for specific problem classes—molecular simulation [1], combinatorial optimization [5], and cryptanalysis [26]—but the current hardware landscape is fragmented. IBM, Google, IonQ, Quantinuum, and Amazon each offer quantum processors with different qubit technologies, gate sets, connectivity topologies, noise profiles, and programming interfaces. A researcher who wants to run a circuit on IBM Quantum must use Qiskit Runtime; the same circuit on Amazon Braket requires a different SDK, different circuit representation, and different result format.

This fragmentation imposes a high barrier to entry. Enterprise customers evaluating quantum computing must hire specialists for each provider, manage multiple credential sets, and write bespoke integration code. Worse, comparing results across providers requires normalizing fundamentally different output formats and noise characteristics.

B. Why Orchestration Matters for Modular Quantum Systems

The industry consensus points toward modular quantum architectures as the path to scale beyond the 1,000-qubit barrier [4], [17]. In a modular system, small high-fidelity quantum modules are connected via photonic or microwave interconnects. This approach avoids the thermal load, frequency crowding, and yield collapse problems that plague monolithic chips at scale.

However, modular architectures create a new software challenge: circuits must be partitioned across modules, inter-module gates must be scheduled using entanglement resources (Bell pairs), and results from distributed execution must be correctly aggregated. No existing cloud quantum platform provides this capability natively.

C. QONTOS's Position

QONTOS occupies the orchestration layer between quantum applications and quantum hardware. The platform is:

- **Software-first:** The full pipeline runs today on simulators; hardware integration is additive, not prerequisite.
- **Hardware-agnostic:** The executor contract (`v1/production/executor_contract.py`) defines a strict interface that any provider adapter must implement. Currently three executors exist: IBM Quantum, Amazon Braket, and Qiskit Aer.
- **Modular-native:** The partitioner, scheduler, and result aggregator are designed from the ground up to handle circuits distributed across multiple quantum modules.

II. Platform Architecture

A. System Overview

QONTOS is organized as a Python monorepo with the structure shown in Table I.

TABLE I
QONTOS REPOSITORY STRUCTURE

Directory	Purpose
`apps/api/`	FastAPI control plane (port 8000)
`apps/worker/`	Celery async orchestration engine
`apps/dashboard/`	Next.js monitoring dashboard (port 3000)
`apps/cli/`	Developer CLI
`services/`	10 microservice-style modules
`packages/schemas/`	Pydantic models (single source of truth)
`packages/config/`	Shared settings, database, auth
`packages/observability/`	Tracing, metrics, span storage
`packages/sdk_python/`	Python SDK (`QontosClient`)
`v1/proprietary/`	15 Q-QOP modules
`v1/enterprise/`	Enterprise error correction, QML
`v1/digital_twin/`	Modular hardware simulator
`benchmarks/`	Benchmark harness
`infra/`	Docker, Kubernetes, Prometheus

B. API Layer

The control plane is a FastAPI application serving 10 route groups under `/api/v1`: health, auth, jobs, runs, backends, projects, admin, sessions, primitives, and utilities. Every response includes correlation headers (`X-Request-ID`, `X-QONTOS-Version`) injected by `CorrelationMiddleware`.

C. Worker Pipeline

The Celery worker implements a 6-stage task chain as shown in Table II.

TABLE II
QONTOS 6-STAGE WORKER PIPELINE

Stage	Task Module	Function
1	`submit_job`	Ingest, partition, schedule, fan-out
2	`execute_partition`	Route to executor, submit
3	`poll_provider`	Async polling for completion
4	`aggregate_results`	Merge into RunResult
5	`finalize_run`	Proof, state update, metrics
6	`retry_failed`	Exponential backoff retry

Tasks use `acks_late=True` for at-least-once delivery semantics. Job state transitions are managed by `JobLifecycle`. Each task binds observability spans via `packages/observability/tracing.py`.

D. Database Model

The PostgreSQL schema defines 11 ORM models organized in a relational hierarchy: `users`, `projects`, `jobs`, `runs`, `partitions`, `scheduled_tasks`, `provider_submissions`, `partial_results`, `run_results`, `audit_records`, and `backend_catalog`. Migrations are managed by Alembic with versioned migration scripts.

The relationship chain follows: User → Project → Job → Run → Partition → ScheduledTask → ProviderSubmission, with `PartialResult` and `RunResult` branching from Run, and `AuditRecord` from Job.

III. CORE PIPELINE

The QONTOS execution pipeline transforms a user-submitted circuit into verified, aggregated results through six stages. Each stage is implemented as an independent service module under `services/`.

A. Circuit Ingestion and Normalization

The `CircuitNormalizer` class is the gateway to the entire pipeline. It accepts circuits in three formats:

- **OpenQASM 2.0/3.0** — parsed via `QuantumCircuit.from_qasm_str()` (QASM 2) or `qiskit.qasm3.loads()` (QASM 3)
- **Qiskit** — native `QuantumCircuit` objects serialized as QASM
- **PennyLane** — JSON tape format, translated via dedicated translator

All formats are normalized into a canonical `CircuitIR` (defined in `packages/schemas/circuit.py`), which captures: `num_qubits`, `num_clbits`, `depth`, `gate_count`, ordered gate list, qubit connectivity, and a deterministic SHA-256 circuit hash.

B. AI-Driven Partitioning

The `Partitioner` class splits a `CircuitIR` into sub-circuits suitable for execution on quantum modules with limited qubit counts:

1) **Graph construction:** Build a `CircuitGraph` where nodes are qubits and edges represent two-qubit gate interactions.
2) **Strategy selection:** Three strategies—`GreedyPartitioner` (fast $O(n)$, for $<$20 qubits), `SpectralPartitioner` (graph Laplacian eigenvector-based [7], for $\geq$20 qubits), and `ManualPartitioner` (user-specified).
3) **Cost evaluation:** Inter-module gate count, communication overhead, partition balance, and cut ratio.
4) **Sub-circuit extraction:** Valid OpenQASM 2.0 strings per partition; inter-module gates tracked separately.

The output is a `PartitionPlan` containing `PartitionEntry` and `DependencyEdge` objects.

C. Capability-Aware Scheduling

The `Scheduler` class assigns each partition to the optimal backend using a weighted multi-criteria scoring model. For each partition–backend pair, `BackendScorer` computes four sub-scores:

- **Fidelity score:** Based on reported 1Q and 2Q gate fidelities
- **Queue depth score:** Inversely proportional to `queue_depth`
- **Cost score:** Normalized cost-per-shot
- **Capacity fit score:** Ratio of partition qubits to backend count

The default `FidelityFirstPolicy` weights fidelity at 0.6, queue at 0.15, cost at 0.1, and capacity at 0.15.

D. Multi-Provider Execution

All executor adapters implement the `ExecutorContract` abstract base class, which mandates six methods as shown in Table III.

TABLE III
EXECUTORCONTRACT INTERFACE METHODS

Method	Purpose
`validate(input)`	Pre-submission validation
`submit(input)`	Submit for execution
`poll(job_id)`	Poll async job status
`cancel(job_id)`	Cancel a running job
`normalize_result()`	Convert to `ExecutorOutput`
`normalize_error()`	Convert to `ExecutorError`

Standardized data types (`ExecutorInput`, `ExecutorOutput`, `ExecutorError`) ensure that no provider-specific logic leaks into the scheduler or API layer.

E. Result Aggregation

The `ResultAggregator` class merges partition results into a unified `RunResult` using one of three strategies:

1) **Passthrough** (single partition): No merging needed.
2) **Independent merge** (tensor product): For partitions with zero inter-module gates. The joint distribution is the Cartesian product of marginals.
3) **Entangled merge** (marginal reconstruction): For partitions with dependency edges. Builds per-partition marginal distributions, averages boundary qubit contributions, and reconstructs the global distribution. A fidelity penalty of 0.02 per cut gate is applied as a conservative classical estimate.

Note: The entangled merge is a classical approximation—it does not recover the full quantum correlations lost by circuit cutting, but produces the best classically-achievable reconstruction from available marginal data.

F. Integrity Verification

`ExecutionHasher` produces deterministic SHA-256 hashes at three levels:

- **Input digest:** Hash of `ExecutionManifest` (job_id, circuit_hash, num_qubits, shots, optimization_level)
- **Execution digest:** Hash of `PartitionPlan` (strategy, partition IDs, qubit assignments, dependencies)
- **Output digest:** Hash of `RunResult` (counts, total_shots, aggregation_method)

The master `proof_hash` is a SHA-256 hash of all three digests combined. This three-layer hash structure ensures that any modification to input, execution decisions, or output is detectable.

IV. PROPRIETARY TECHNOLOGY STACK (Q-QOP)

The QONTOS Quantum Orchestration Platform (Q-QOP) comprises 15 proprietary modules. Each module addresses a specific technical challenge in modular quantum computing.

A. Q-SCALE: Scalable Quantum Architecture

Q-SCALE models the QONTOS hierarchical modular architecture across four tiers. The `hierarchy.py` module defines core dataclasses for `QPUChiplet`, `QuantumModule`, `QuantumSystem`, and `QONTOSDataCenter`, while the current research program maps those classes into base, aggressive, and stretch scenarios. Additional modules model thermal budgets, frequency collision analysis, photonic interconnect simulation, live module hot-swap, cross-module qubit allocation, and cryogenic control optimization.

B. Q-SHIELD: Adaptive Error Correction

Q-SHIELD is an adaptive error correction engine. The `AdaptiveCodeSelector` evaluates surface codes, bivariate bicycle (BB) qLDPC codes, color codes, and concatenated codes, switching between them as measured error rates change. The `QONTOSSyndromeDecoder` implements a hybrid belief-propagation + neural decoder architecture. `LogicalQubitPool` manages logical qubits with `MagicStateFactory` for T-gate distillation. `InterModuleEC` handles distributed syndrome extraction across photonic interconnects.

C. Q-MIND: AI Engine

Q-MIND replaces static heuristics with learned optimization. The `IntelligentPartitioner` evaluates partition plans based on historical execution data. The `PredictiveScheduler` forecasts backend availability using time-series models. Additional components provide noise-aware compilation, automatic error mitigation strategy selection, reinforcement learning for circuit optimization [12], Bayesian optimization for VQE landscapes, causal diagnostics, transfer learning, and active experiment design.

D. Q-CONNECT: Photonic Interconnect Orchestration

Q-CONNECT orchestrates the photonic interconnect fabric for multi-module execution, modeling optical interconnects with transducer specifications, fiber channel properties, and end-to-end link performance. The `EntanglementOrchestrator` pre-schedules Bell pair generation based on circuit dependency analysis. Additional components handle quantum teleportation protocols [17], distributed circuit compilation, network tomography, and Universal Blind Quantum Computing (UBQC) [3].

E. Q-FORGE: Domain-Specific Applications

Q-FORGE provides domain-specific quantum application frameworks including molecular Hamiltonian generation via Jordan–Wigner transformation, ADAPT-VQE [12], active space selection, drug discovery pipelines, materials simulation, portfolio optimization, Monte Carlo amplification, and combinatorial optimization (MaxCut, TSP, vehicle routing, job-shop scheduling) via QAOA [5].

F. Q-PULSE: Pulse-Level Quantum Control

Q-PULSE provides pulse-level control for superconducting processors, including Gaussian/FlatTop/CosineRise pulse synthesis with DRAG optimization, cross-resonance gate calibration [25], multiplexed dispersive readout, dynamical decoupling sequences (XY4, CPMG, UDD, KDD), and cross-module clock synchronization with transduction pulse design.

G. Q-VERIFY: Verification and Validation

Q-VERIFY provides mathematical guarantees for quantum computation correctness through circuit equivalence checking, state certification via entanglement witnesses, gate fidelity certification via randomized benchmarking, Quantum Volume certification, certified random number generation via Bell inequality violation, and cross-module computation verification.

H. Q-TENSOR: Tensor Network Simulation

Q-TENSOR enables simulation beyond statevector limits via arbitrary-rank tensor operations, Matrix Product States (MPS) [28] with bond dimension up to 4096 (enabling 1000+ qubit simulation for bounded-entanglement circuits), Matrix Product Operators [24], DMRG [29] for variational ground-state search, and full circuit simulation via MPS evolution. The entire implementation is pure NumPy.

I. Additional Q-QOP Modules

- **Q-GENESIS:** Chip design, device physics [14], fabrication modeling, and automated calibration.
- **Q-SECURE:** Zero-knowledge proofs [16], UBQC [3], post-quantum cryptography (NIST ML-KEM [22]), and Merkle-tree audit trails.
- **Q-FABRIC:** Distributed quantum operating system with process scheduling, qubit memory management, fault-tolerant gate scheduling, and performance modeling.
- **Q-BRIDGE:** Universal hardware abstraction layer with 7 backend adapters (IBM, Google, IonQ, Quantinuum, Rigetti, Xanadu, QONTOS Modular) and automatic gate set translation.
- **Q-CHRONO:** Real-time calibration drift tracking [18], adaptive recalibration, TLS defect modeling, and predictive maintenance.
- **Q-ORACLE:** Quantum-computing-specific knowledge graph with ontology, reasoning engine, and experiment knowledge base.
- **Q-AGENT:** Autonomous experiment system with hierarchical task planning, multi-agent coordination, self-driving VQE/QAOA, and natural-language goal translation.

V. Modular Hardware Model

A. Scenario-Based Hierarchy

QONTOS's hardware model defines a 4-tier hierarchy interpreted through base, aggressive, and stretch scenarios (Table IV).

The current stretch reference across the paper set is: 2,000 physical qubits per chiplet, 5 chiplets per module, 10 modules per system, and 10 systems per data center—yielding the million-qubit target.

TABLE IV
QONTOS 4-Tier Modular Architecture Scenarios

Tier	Unit	Base	Aggr.	Stretch
1	QPU Chiplet	5–200	500–2k	2,000
2	Module	400–2.5k	10k	10,000
3	System	10k–20k	100k	100,000
4	Data Center	early	100k–1M	1,000,000

B. Digital Twin Simulation

The digital twin (`v1/digital_twin/modular_simulator.py`) simulates workloads on modular architecture candidates. For a given `SystemConfig`, `simulate_workload()` estimates: total gate count, circuit fidelity, runtime, Bell pairs required, and effective circuit depth increase from inter-module serialization.

C. Transduction Scenario Bands

The digital twin groups interconnect assumptions into scenario bands:

- $\geq 20\%$: Stretch band (full modular planning)
- $\geq 10\%$: Aggressive band (meaningful multi-module operation)
- 1–10%: Base/early band (staged modular validation)
- $< 1\%$: Research-limited (device and link R&D)

D. Comparison with Monolithic Approaches

The modular architecture addresses three scaling pressures: (1) thermal load distribution across independent cryogenic envelopes, (2) frequency crowding localized to chiplets/modules, and (3) yield pressure shifted toward smaller repeated units.

VI. Error Correction and Mitigation

A. Error Mitigation (NISQ Era)

The `ErrorMitigator` class provides three industry-standard mitigation techniques:

1) **Zero-Noise Extrapolation (ZNE):** Execute at multiple noise amplification factors, extrapolate to zero-noise limit [27].
2) **Probabilistic Error Cancellation (PEC):** Quasi-probability decomposition of ideal operations into noisy implementable operations [27].
3) **Measurement Mitigation (M3-style):** Calibration matrix from basis state measurements, inverse applied to correct readout errors [19].

B. Surface Codes

The `SurfaceCode` class implements the rotated planar surface code. The logical error rate follows [11]:

$$p_L \sim 0.03 \cdot \left(\frac{p}{p_{\text{th}}}\right)^{(d+1)/2} \tag{1}$$

where $p_{\text{th}} \approx 1.0\%$ and d is the code distance. At physical error rate $p = 0.1\%$: $d = 3$: 2.7×10^{-4}, $d = 5$: 2.7×10^{-6}, $d = 7$: 2.7×10^{-8}.

C. Low-Overhead Code Families

QONTOS implements the bivariate bicycle qLDPC code family [2] as shown in Table V.

TABLE V
BIVARIATE BICYCLE qLDPC CODE PARAMETERS

Code	Phys.	Log.	Dist.	Ratio
$[[72, 12, 6]]$	72	12	6	6:1
$[[144, 12, 12]]$	144	12	12	12:1
$[[288, 12, 18]]$	288	12	18	24:1

The broader overhead research program targets: 1000:1 (conservative baseline), 300:1 (aggressive target), and 100:1 (stretch target).

D. Adaptive Code Switching

Q-SHIELD's `AdaptiveCodeSelector` monitors real-time error rates and selects codes dynamically: surface code at low distance for $p < 0.1\%$, qLDPC $[[144, 12, 12]]$ for 0.1–0.5%, concatenated codes for 0.5–1%, and color codes near threshold for transversal gate advantages.

VII. BENCHMARK RESULTS

All benchmarks are executed through the QONTOS pipeline using the `BenchmarkRunner` class. **All measurements below were obtained on Qiskit Aer simulator (noiseless statevector backend).** Hardware QPU measurements are pending Phase 1 chiplet delivery.

A. Benchmark Suite

TABLE VI
QONTOS BENCHMARK SUITE

Benchmark	Circuit	Qubits	Threshold
Bell Pair	H + CNOT	2	≥ 0.85
GHZ-3	H + 2 CNOT	3	≥ 0.85
GHZ-5	H + 4 CNOT	5	≥ 0.85
QFT-4	H + CU1 + SWAP	4	≥ 0.85
Bernstein–Vazirani	H + CNOT oracle	4	≥ 0.85
H_2 VQE Ansatz	RY + CNOT + RY	2	≥ 0.85
Random 5Q	Mixed (depth=10)	5	≥ 0.85

B. Methodology

Each circuit is executed through the full pipeline: (1) normalization via `CircuitNormalizer`, (2) execution via `LocalSimulatorExecutor`, (3) fidelity computation. Fidelity is defined as:

$$F = \frac{\sum_{\text{expected states}} \text{counts}_i}{\text{total_shots}} \tag{2}$$

Pass threshold is 0.85; default shot count is 8,192.

C. Results (Noiseless Simulator)

On a noiseless simulator, all benchmarks achieve fidelity ≥ 0.99. These results validate pipeline correctness—the ingest-normalize-execute path preserves circuit semantics. The benchmarks detect regressions in the normalization and execution path, not hardware noise characteristics.

VIII. SCALING PROJECTIONS

The QONTOS scaling roadmap is a gated program with three scenario bands: Base, Aggressive, and Stretch.

A. Program Phases Through 2030

TABLE VII
QONTOS GATED DEVELOPMENT ROADMAP

Phase	Years	Key Outcomes (Base / Aggr. / Stretch)
Foundation	2025–26	Platform + benchmark harness / First HW validation / Stretch device evidence
Sputnik	2026–27	Small modular HW / 10k-qubit module assumptions / Stretch module target evidence
Pioneer	2027–28	Distributed runtime / Multi-module demos / 100k physical qubit path
Horizon	2028–29	Modular platform / 100k phys. + 100–1k logical / 500k phys. + 5k logical
Summit	2029–30	Commercial platform / Large FT machine / 1M phys. + 10k logical

B. Risk and Gate Logic

Each phase carries explicit gate logic:

- **Foundation:** Platform and literature assumptions must be reproducible.
- **Sputnik:** Packaging, thermal, and first logical-qubit path must stay credible.
- **Pioneer:** Communication and decoder path must support multi-module operation.
- **Horizon:** Aggressive QEC and interconnect assumptions must improve materially.
- **Summit:** Stretch case advances only if prior gates pass across devices, QEC, interconnects, and cryogenics.

IX. SECURITY AND VERIFICATION

A. Zero-Knowledge Proofs

QONTOS implements zero-knowledge proof protocols for computation verification:

- **ZKQuantumProof:** Adapted Mahadev protocol [16] with commit-challenge-respond framework (128+ rounds; cheating detected with probability $\geq 1 - 2^{-n}$).
- **NonInteractiveProof:** Fiat–Shamir transform [6] deriving challenges from transcript hash (SHA3-256).
- **VerifiableDelegation:** Trap-qubit-based delegated computation [8] with one-time-pad encryption and qubit permutation (25% trap qubits).

B. Universal Blind Quantum Computing

UBQC [3] is implemented in two modules: Q-CONNECT (core protocol) and Q-SECURE (enterprise wrapper with `ClientServerProtocol`).

C. Quantum-Safe TLS

`HybridTLS` implements hybrid classical/post-quantum TLS using lattice-based key encapsulation (NIST FIPS 203, ML-KEM [22]), stateless hash-based signatures (NIST FIPS 205), and runtime crypto agility.

D. SHA-256 Execution Integrity

Every execution produces a three-layer SHA-256 hash chain: input digest → execution digest → output digest → master proof hash. The chain is stored in `run_results.proof_hash` and anchored to the Q-SECURE audit blockchain.

X. Observability and Operations

A. Correlation ID Tracing

Every API request receives a unique `X-Request-ID` propagated through the Celery task chain as `trace_id`. The `Tracer` class creates hierarchical spans:

```
trace_id: "abc123"
  |-- span: submit_job
  |     |-- span: circuit_normalize
  |     |-- span: partition
  |     |-- span: schedule
  |-- span: execute_partition (parallel)
  |-- span: aggregate_results
  |-- span: finalize_run
```

Each span records: `span_id`, `trace_id`, `parent_span_id`, `service_name`, `operation`, timing, status, and arbitrary attributes.

B. Pipeline Metrics and Prometheus

`PipelineMetrics` tracks job submission counts, per-stage timing, error counters, and backend utilization. Prometheus scrapes the `/metrics` endpoint with 30-day retention. Grafana dashboards visualize throughput, latency percentiles, and error rates.

XI. Infrastructure

A. Docker Compose Deployment

The standard deployment defines 8 services (Table VIII).

TABLE VIII
QONTOS Docker Compose Services

Service	Image	Port	Purpose
api	Custom	8000	FastAPI control plane
worker	Custom	—	Celery engine
dashboard	Custom	3000	Next.js UI
postgres	postgres:16	5432	Primary database
redis	redis:7	6379	Broker + cache
minio	minio/minio	9000	Object storage
prometheus	prom/prometheus	9090	Metrics
grafana	grafana/grafana	3001	Dashboards

All services communicate over a dedicated `qontos` bridge network with configured health checks.

B. CI/CD Pipeline

The CI pipeline runs 7 jobs on every push: lint (Ruff), typecheck (mypy), unit tests (pytest + coverage), schema validation, integration tests (live Postgres + Redis), benchmarks, container build, and security scan (pip-audit).

XII. Competitive Analysis

Table IX summarizes QONTOS's positioning against major quantum platforms based on publicly available information as of March 2026.

QONTOS structural advantages: (1) hardware-agnostic orchestration with cross-provider partitioning, scheduling, and aggregation; (2) modular-native software designed from day one for distributed quantum execution; (3) cryptographic execution proofs; (4) adaptive error correction via Q-SHIELD; (5) full-stack proprietary technology (15 Q-QOP modules).

XIII. Roadmap and Future Work

A. Near-Term (2026)

Strengthen the software-first orchestration platform, keep benchmark and replay evidence reproducible, connect digital-twin assumptions to the modular research stack, and prepare the first hardware-linked validation path.

B. Medium-Term (2027–2028)

Mature the distributed runtime and modular benchmark program, validate packaging/thermal/interconnect assumptions at module scale, establish the first logical-qubit milestone path on modular hardware, and connect platform progress with hardware integration via the benchmark ladder.

C. Long-Term (2029–2030)

Pursue aggressive-scale modular systems with meaningful logical resources, validate whether stretch assumptions can land together, and use flagship benchmark progress as the final proof layer.

XIV. Conclusion

QONTOS is an implementation-grounded quantum orchestration platform with a clear software foundation and a modular research program built around the same architecture thesis. The core pipeline—circuit ingestion, AI-driven partitioning, capability-aware scheduling, multi-provider execution, result aggregation, and integrity verification—is implemented, tested, and benchmarked on Qiskit Aer simulators. The broader Q-QOP stack extends that platform into digital-twin modeling, modular architecture studies, error correction, interconnect research, and verification tooling.

The platform's key differentiator is its modular-native design. The partitioner, scheduler, aggregator, and digital twin all reason natively about multi-module topologies, inter-module communication costs, and scenario-based performance envelopes.

The strongest current QONTOS message is: the software-first orchestration platform exists today; the modular architecture is backed by an organized research program; and the largest hardware and logical-qubit goals belong to the aggressive and stretch scenarios, with clear gates and benchmark evidence required at each step.

TABLE IX
COMPETITIVE POSITIONING (BASED ON PUBLIC INFORMATION, MARCH 2026)

Dimension	IBM Quantum	Google Cirq	Amazon Braket	IonQ/Quantinuum	QONTOS
Architecture	Monolithic	Monolithic	Multi-provider	Trapped ion	Modular
Multi-provider	IBM only	Google only	Yes	Provider SDKs	Yes (extensible)
Circuit partitioning	No	No	No	No	Built-in
Execution proofs	No	No	No	No	SHA-256 chain
Error correction	Research demos	Demonstrated	N/A	Research	Adaptive multi-code
Orchestration	Session-based	Manual	Direct submit	Direct submit	Full pipeline

REFERENCES

[1] A. Aspuru-Guzik, A. D. Dutoi, P. J. Love, and M. Head-Gordon, "Simulated quantum computation of molecular energies," *Science*, vol. 309, pp. 1704–1707, 2005.

[2] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, pp. 778–782, 2024.

[3] A. Broadbent, J. Fitzsimons, and E. Kashefi, "Universal blind quantum computation," in *Proc. FOCS*, 2009.

[4] C. Chamberland *et al.*, "Building a fault-tolerant quantum computer using concatenated cat codes," *PRX Quantum*, vol. 3, p. 010329, 2022.

[5] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[6] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Proc. CRYPTO*, 1986.

[7] M. Fiedler, "Algebraic connectivity of graphs," *Czechoslovak Mathematical Journal*, vol. 23, pp. 298–305, 1973.

[8] J. Fitzsimons and E. Kashefi, "Unconditionally verifiable blind quantum computation," *Phys. Rev. A*, vol. 96, p. 012303, 2017.

[9] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Phys. Rev. A*, vol. 86, p. 032324, 2012.

[10] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[11] Google Quantum AI, "Quantum error correction below the surface code threshold," *Nature*, 2024.

[12] H. R. Grimsley, S. E. Economou, E. Barnes, and N. J. Mayhall, "An adaptive variational algorithm for exact molecular simulations on a quantum computer," *Nature Commun.*, vol. 10, p. 3007, 2019.

[13] S. Halevi and S. Micali, "Practical and provably-secure commitment schemes from collision-free hashing," in *Proc. CRYPTO*, 1996.

[14] J. Koch *et al.*, "Charge-insensitive qubit design derived from the Cooper pair box," *Phys. Rev. A*, vol. 76, p. 042319, 2007.

[15] J. Lee *et al.*, "Even more efficient quantum computations of chemistry through tensor hypercontraction," *PRX Quantum*, vol. 2, p. 030305, 2021.

[16] U. Mahadev, "Classical verification of quantum computations," in *Proc. FOCS*, 2018.

[17] C. Monroe *et al.*, "Large-scale modular quantum-computer architecture with atomic memory and photonic interconnects," *Phys. Rev. A*, vol. 89, p. 022317, 2014.

[18] C. Mueller *et al.*, "Towards understanding two-level-systems in amorphous solids: Insights from quantum circuits," *Rep. Prog. Phys.*, vol. 82, p. 124501, 2019.

[19] P. D. Nation, H. Kang, N. Sundaresan, and J. M. Gambetta, "Scalable mitigation of measurement errors on quantum computers," *PRX Quantum*, vol. 2, p. 040326, 2021.

[20] R. Orús, "A practical introduction to tensor networks," *Ann. Phys.*, vol. 349, pp. 117–158, 2014.

[21] P. Panteleev and G. Kalachev, "Asymptotically good quantum and locally testable classical LDPC codes," in *Proc. STOC*, 2022.

[22] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *Proc. STOC*, 2005.

[23] M. Reiher, N. Wiebe, K. M. Svore, D. Wecker, and M. Troyer, "Elucidating reaction mechanisms on quantum computers," *Proc. Natl. Acad. Sci.*, vol. 114, pp. 7555–7560, 2017.

[24] U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states," *Ann. Phys.*, vol. 326, pp. 96–192, 2011.

[25] S. Sheldon *et al.*, "Procedure for systematically tuning up cross-talk in the cross-resonance gate," *Phys. Rev. A*, vol. 93, p. 060302(R), 2016.

[26] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. FOCS*, 1994.

[27] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," *Phys. Rev. Lett.*, vol. 119, p. 180509, 2017.

[28] G. Vidal, "Efficient simulation of slightly entangled quantum computations," *Phys. Rev. Lett.*, vol. 91, p. 147902, 2003.

[29] S. R. White, "Density matrix formulation for quantum renormalization groups," *Phys. Rev. Lett.*, vol. 69, p. 2863, 1992.